

Aggregates are named tuples

“Call him Voldemort, Harry. Always use the proper name for things.”

— J.K. Rowling, *Harry Potter and the Sorcerer's Stone*

I. Quick Introduction

In C++ we have:

- tuples - types that provide access to members by index. Those are useful for generic programming
- aggregates - types with named fields. Those are just easy to use.

This paper was inspired by multiple years of experience with [PFR/magic_get library](#). The core idea of this paper is to add functionality to some aggregates so that they could behave as tuples.

II. Motivation and Examples

`std::tuple` and `std::pair` are great for generic programming, however they have disadvantages. First of all, code that uses them becomes barely readable. Consider two definitions:

```
struct auth_info_aggregate {
    std::int64_t id;
    std::int64_t session_id;
    std::int64_t source_id;
    std::time_t valid_till;
};

using auth_info_tuple = std::tuple<
    std::int64_t,
    std::int64_t,
    std::int64_t,
    std::time_t
>;
```

Definition via structure is much more clear. Same story with usages: `return std::get<1>(value);` vs. `return value.session_id;`

Another advantage of aggregates is a more efficient copy, move construction and assignments:

```
template <class T>
constexpr bool validate() {
    static_assert(std::is_trivially_move_constructible_v<T>);
    static_assert(std::is_trivially_copy_constructible_v<T>);
    static_assert(std::is_trivially_move_assignable_v<T>);
    static_assert(std::is_trivially_copy_assignable_v<T>);
    return true;
}

constexpr bool tuples_fail = validate<auth_info_tuple>(); // Fails majority of checks
constexpr bool aggregates_are_ok = validate<auth_info_aggregate>();
```

Because of the above issues many coding guidelines **recommend to use aggregates instead of tuples**.

However at the moment aggregates fail when it comes to the functional like programming:

```
namespace impl {
    template <class Stream, class Result, std::size_t... I>
    void fill_files(Stream& s, Result& res, std::index_sequence<I...>) {
        (s >> ... >> std::get<I>(res));
    }
}

template <class T>
T ExecuteSQL(std::string_view statement) {
    std::stringstream stream;
    // some logic that feeds data into stream

    T result;
    impl::fill_files(stream, result, std::make_index_sequence<std::tuple_size_v<T>>());
    return result;
}

constexpr std::string_view query = "SELECT id, session_id, source_id, valid_till FROM auth";
const auto tuple_result = ExecuteSQL<auth_info_tuple>(query);
const auto aggregate_result = ExecuteSQL<auth_info_aggregate>(query); // does not compile
```

// Playground <https://godbolt.org/z/y491ya>

By bringing the functionality of tuples into aggregates we get all the advantages of tuples without loosing advantages of aggregates. We get **named tuples**.

III. The Idea

Make `std::get`, `std::tuple_element` and `std::tuple_size` work with aggregates that have no base classes. This also makes `std::tuple_element_t`, `std::tuple_size_v`, `std::apply` and `std::make_from_tuple` usable with aggregates.

IV. Interaction with other papers

[P1061 "Structured Bindings can introduce a Pack"](#) makes it really simple to implement the ideas proposed in this paper. For example `std::tuple_size` could be implemented as:

```
template <class T>
constexpr std::size_t fields_count() {
    auto [...x] = T();
    return sizeof...(x);
}

template <class T>
struct tuple_size: std::integral_constant<std::size_t, fields_count<T>()> {};
```

[P1061](#) is not a requirement for this paper acceptance. Same logic could be implemented as a compiler built-in or even via some metaprogramming tricks, as in [PFR/magic_get_library](#).

There may be concerns, that proposed functionality may hurt [N4818 "C++ Extensions for Reflection"](#) adoption, as some of functionality becomes available without reflection. Experience with [PFR/magic_get_library](#) shows that `std::get` and `std::tuple_size` functions cover only very basic cases of reflection. We still need reflection for trivial things, like serialization to JSON, because only reflection gives us field names of the structure.

Parts of [P1858R1 "Generalized pack declaration and usage"](#) address some of the ideas of this paper on a language level and give simple to use tools to implement ideas of this paper. However this paper brings capabilities symmetry to the standard library, shows another approach to deal with field access by index and allows existing user code to work out-of-the-box with aggregates:

C++20	This paper	P1858
<pre>// Works only with tuples // int foo(auto value) { if (!std::get<10>(value)) { return 0; } return std::apply(function, value); }</pre>	<pre>// Works with tuples and aggregates // No code change required int foo(auto value) { if (!std::get<10>(value)) { return 0; } return std::apply(function, value); }</pre>	<pre>// Works with tuples and aggregates // Users are forced to rewrite code int foo(auto value) { if (!value::[10]) { return 0; } return std::invoke(function, value }</pre>
<pre>template <class T> auto portable_function(const T& value) { // Works with tuples since C++11 return std::get<2>(value); }</pre>	<pre>template <class T> auto portable_function(const T& value) { // Works with tuples since C++11 and with aggregates return std::get<2>(value); }</pre>	<pre>template <class T> auto portable_function(const T& value) #ifdef __cpp_generalized_packs // Works with tuples and aggregate return value::[2]; #else // Works with tuples since C++11 return std::get<2>(value); #endif }</pre>

V. Wording

Adjust `[tuple.syn]`:

```
// [tuple.helper], tuple helper classes
template<class T> struct tuple_size; // not defined
template<class T> struct tuple_size<const T>;

template<class T>
concept decomposable = see-below; // exposition only

template<decomposable T> struct tuple_size;

template<class... Types> struct tuple_size<tuple<Types...>>;

template<size_t I, class T> struct tuple_element; // not defined
template<size_t I, class T> struct tuple_element<I, const T>;

template<size_t I, decomposable T> struct tuple_element;

template<size_t I, class... Types>
struct tuple_element<I, tuple<Types...>>;
```

```

template<size_t I, class T>
    using tuple_element_t = typename tuple_element<I, T>::type;

// [tuple.elem], element access
template<size_t I, decomposable T>
    constexpr tuple_element_t<I, T>& get(T&) noexcept;
template<size_t I, decomposable T>
    constexpr tuple_element_t<I, const T>& get(const T&) noexcept;
template<size_t I, class... Types>
    constexpr tuple_element_t<I, tuple<Types...>& get(tuple<Types...>&) noexcept;
template<size_t I, class... Types>
    constexpr tuple_element_t<I, tuple<Types...>&& get(tuple<Types...>&&) noexcept;
template<size_t I, class... Types>
    constexpr const tuple_element_t<I, tuple<Types...>& get(const tuple<Types...>&) noexcept;
template<size_t I, class... Types>
    constexpr const tuple_element_t<I, tuple<Types...>&& get(const tuple<Types...>&&) noexcept;
template<class T, class... Types>
    constexpr T& get(tuple<Types...>& t) noexcept;
template<class T, class... Types>
    constexpr T&& get(tuple<Types...>&& t) noexcept;
template<class T, class... Types>
    constexpr const T& get(const tuple<Types...>& t) noexcept;
template<class T, class... Types>
    constexpr const T&& get(const tuple<Types...>&& t) noexcept;

```

Choose one of the alternatives for *decomposable* definition:

1. `template<class T> concept decomposable;`
Satisfied if T is an aggregate without base class.
2. `template<class T> concept decomposable;`
Satisfied if T is an aggregate.
3. `template<class T> concept decomposable;`
Satisfied if the expression `auto [arg0, ..., argi] = std::declval<T>();` is well formed for some amount of *arg* arguments.

Adjust [tuple.helper]:

```

template<class T> struct tuple_size;
    All specializations of tuple_size meet the Cpp17UnaryTypeTrait requirements ([meta.rqmts]) with a base
    characteristic of integral_constant<size_t, N> for some N.

```

====INSERT ONE OF THE decomposable DEFINITIONS====

```

template<decomposable T> struct tuple_size;
    Let N denote the fields count in T. Specialization meets the Cpp17UnaryTypeTrait
    requirements ([meta.rqmts]) with a base characteristic of integral_constant<size_t, N>.

```

```

template<class... Types>
struct tuple_size<tuple<Types...>> : public integral_constant<size_t, sizeof...(Types)> { };

```

....

```

template<size_t I, decomposable T> struct tuple_element;
    Let TE denote the type of the Ith field of T, where indexing is zero-based. Specialization meets the Cpp17TransformationTrait
    requirements ([meta.rqmts]) with a member typedef type that names the type TE.

```

Add paragraph at the beginning of [tuple.elem]:

Element access [tuple.elem]

```

template<size_t I, decomposable T>
    constexpr tuple_element_t<I, T>& get(T& t) noexcept;
template<size_t I, decomposable T>
    constexpr tuple_element_t<I, const T>& get(const T& t) noexcept;
    Mandates: I < tuple_size_v<T>.
    Returns: A reference to the Ith field of t, where indexing is zero-based.

```

VI. Acknowledgements

Many thanks to Barry Revzin for writing P1858 and providing early notes on this paper.